

ORION



Justification des choix techniques **Projet MDD**



Auteur : **ALSABR IBRAHIM**

Version 0.0.1

Aperçu / Synthèse.....	2
Choix techniques.....	3
Choix 1	3
Choix 2	4
Choix 3	4
Choix 4	5
Choix 5	5
Choix 6	6
Choix 7	6
Choix 8	7
Choix 9	7
Choix 10	8

Aperçu / Synthèse

Le projet MDD est une application web full-stack de type réseau social autour de la programmation. L'architecture retenue est une **API REST stateless** côté backend, consommée par une **SPA (Single Page Application)** côté frontend.

Backend : Java 21 avec Spring Boot 4, base de données MySQL, sécurisation via JWT (OAuth2 Resource Server), persistance via Spring Data JPA / Hibernate. Le code est structuré en couches strictes (Controller → Service → Repository → Model) avec le pattern DTO pour isoler les représentations réseau du domaine métier.

Frontend : Angular 19 en mode standalone avec TypeScript 5.7, routing child routes, formulaires ngModel, et communication HTTP sécurisée via un intercepteur injectant automatiquement le token JWT dans chaque requête. Tous les choix respectent les contraintes imposées : architecture REST, authentification par token, séparation frontend/backend, responsive design.

Choix techniques

Choix 1 — Spring Boot 4 (Java 21)

<i>choix technique</i>	<i>lien vers le site / la documentation / une ressource</i>	<i>but du choix</i>
Spring Boot 4.0.3 + Java 21	https://docs.spring.io/spring-boot/index.html	Framework principal du backend

Justification : Spring Boot est le standard pour les API REST Java. Il configure automatiquement le serveur et les dépendances, ce qui permet de démarrer rapidement sans configuration manuelle complexe.

Choix 2 — Spring Security + JWT

<i>choix technique</i>	<i>lien vers le site / la documentation / une ressource</i>	<i>but du choix</i>
Spring Security 6 + OAuth2 Resource Server	https://docs.spring.io/spring-security/reference/	Sécurisation de l'API
BCryptPasswordEncoder	https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html	Hachage des mots de passe

Justification : Le JWT permet une authentification stateless : le serveur n'a pas besoin de stocker les sessions. BCrypt protège les mots de passe stockés en base contre les attaques par force brute. La contrainte impose que la connexion persiste entre sessions — le JWT côté client remplit exactement ce rôle.

Choix 3 — Spring Data JPA + MySQL

<i>choix technique</i>	<i>lien vers le site / la documentation / une ressource</i>	<i>but du choix</i>
Spring Data JPA + Hibernate 6	https://docs.spring.io/spring-data/jpa/reference/	Persistence des données
MySQL 8	https://dev.mysql.com/doc/	Base de données relationnelle

Justification : Les données (utilisateurs, articles, abonnements, commentaires) ont des relations fortes entre elles. JPA mappe ces relations Java ↔ SQL

automatiquement via des annotations, sans écrire de requêtes SQL manuelles. MySQL est fiable et s'intègre nativement avec Spring Boot.

Choix 4 — Pattern DTO

<i>choix technique</i>	<i>lien vers le site / la documentation / une ressource</i>	<i>but du choix</i>
Data Transfer Objects (DTO)	https://martinfowler.com/eaCatalog/dataTransferObject.html	Architecture — isolation couche API / domaine

Justification : Les entités JPA ne sont jamais envoyées directement en réponse HTTP. Des DTO dédiés contrôlent exactement ce qui est exposé, évitent les problèmes de sérialisation Hibernate, et permettent de valider les données entrantes avec @NotBlank, @Size, @Email.

Choix 5 — Lombok

<i>choix technique</i>	<i>lien vers le site / la documentation / une ressource</i>	<i>but du choix</i>
Project Lombok	https://projectlombok.org/features/	Réduction du code répétitif Java

Justification : Lombok génère automatiquement les getters, setters et constructeurs via des annotations (@Data, @RequiredArgsConstructor). Cela rend les classes Java beaucoup plus courtes et lisibles, sans impact sur les performances

Choix 6 — SpringDoc OpenAPI (Swagger UI)

<i>choix technique</i>	<i>lien vers le site / la documentation / une ressource</i>	<i>but du choix</i>
springdoc-openapi 2.8.13	https://springdoc.org/	Documentation interactive de l'API

Justification : SpringDoc génère automatiquement une documentation visuelle de tous les endpoints à partir du code existant. L'interface Swagger UI permet de tester l'API directement depuis le navigateur, sans outil externe.

Choix 7 — Angular 19 (Standalone + Signals)

<i>choix technique</i>	<i>lien vers le site / la documentation / une ressource</i>	<i>but du choix</i>
Angular 19.1 + TypeScript 5.7	https://angular.dev/	Framework principal frontend

Justification : Angular impose une structure claire adaptée à une SPA de cette taille. Les standalone components simplifient la déclaration des dépendances. Les Signals gèrent l'état local des composants de façon simple et performante. TypeScript garantit la cohérence des types entre le backend et le frontend.

Choix 8 — Angular Router (Child Routes)

<i>choix technique</i>	<i>lien vers le site / la documentation / une ressource</i>	<i>but du choix</i>
@angular/router	https://angular.dev/guide/routing	Navigation SPA avec layout partagé

Justification : Les child routes permettent au composant HomePage d'afficher le header une seule fois, pendant que les pages (articles, thèmes, profil) s'affichent dans le <router-outlet> en dessous. L'authGuard redirige vers /login si l'utilisateur n'est pas connecté.

Choix 9 — Angular HttpClient + Intercepteur JWT

<i>choix technique</i>	<i>lien vers le site / la documentation / une ressource</i>	<i>but du choix</i>
HttpClient + HttpInterceptorFn	https://angular.dev/guide/http/interceptors	Communication API sécurisée

Justification : L'intercepteur ajoute automatiquement le token JWT (Authorization: Bearer ...) à chaque requête HTTP. Cela évite de répéter cette logique dans chaque service, et garantit qu'aucun appel ne part sans authentification.

Choix 10 — RxJS

<i>choix technique</i>	<i>lien vers le site / la documentation / une ressource</i>	<i>but du choix</i>
RxJS 7.8	https://rxjs.dev/	Gestion des appels HTTP asynchrones

Justification : Angular HttpClient retourne des Observable (RxJS), ce qui est le standard de l'écosystème Angular. RxJS permet de gérer facilement les succès et erreurs des appels API via `subscribe({ next, error })`.